AxiomGraph: A Software Framework for AI-Agent-Architected, Self-Evolving Cognitive Substrates

Abstract

This paper introduces AxiomGraph, a software framework for creating AI agents that can architect, monitor, modify, and evolve their own cognitive structures through a dynamic graph of nodes and synapses. The framework is inspired by biological development, neuromorphic computing, memristive systems, physical reservoir computing, and the companion concept of solution-grown agent-specific neuromorphic substrates. However, AxiomGraph removes the immediate need for physical fabrication by implementing the same developmental principles in a purely digital environment.

In AxiomGraph, each AI agent begins with a digital genome: a structured specification defining its purpose, cognitive modules, memory systems, tool interfaces, growth constraints, pruning rules, safety boundaries, and performance metrics. The agent's cognition is represented as a living graph. Nodes represent reasoning modules, memory systems, skills, tools, subagents, evaluators, simulators, or domain-specific processors. Edges represent synaptic pathways with dynamic properties such as routing strength, trust, cost, latency, success rate, decay, plasticity, and specialization.

The central claim of this paper is that AI agents should not remain static prompt-driven systems. They should become developmental cognitive systems capable of growing new capabilities, strengthening successful reasoning pathways, pruning ineffective ones, and generating individualized internal architectures over time. This creates a bridge between software agents, neuromorphic principles, adaptive memory, and future physical embodiments such as 3D-printed solution-grown neuromorphic substrates.

1. Introduction

Current AI agents are typically assembled from a static combination of model prompts, tools, memory stores, vector databases, workflows, and API integrations. While such systems can be powerful, they are often architecturally rigid. Their internal organization does not naturally develop in response to use. A customer-service agent, research agent, coding agent, or ecological modeling agent may accumulate memory, but the structural relationship between its modules usually remains externally designed.

Biological intelligence is different. Human brains are not simply loaded with fixed programs. They develop through growth, reinforcement, pruning, specialization, and experience. The structure of the brain reflects both genetic constraints and lived interaction.

AxiomGraph applies this developmental principle to software agents.

Instead of asking: How do we prompt an AI agent better?

AxiomGraph asks: How does an AI agent grow a better cognitive architecture over time?

This distinction is critical. Prompt engineering changes the instruction layer. Fine-tuning changes model weights. Retrieval systems add external memory. AxiomGraph changes the agent's cognitive topology.

2. Core Thesis

The central thesis is:

An AI agent can be represented as a self-evolving cognitive graph whose architecture develops through task interaction, feedback, reinforcement, pruning, and self-reflection.

This framework allows an agent to:

1. Represent its own cognitive architecture.
2. Monitor which cognitive pathways succeed or fail.
3. Strengthen useful pathways.
4. Weaken or prune ineffective pathways.
5. Grow new specialized modules.
6. Reorganize itself around emerging tasks.
7. Preserve an auditable developmental history.
8. Generate future physical neuromorphic substrate designs.

The framework is called AxiomGraph because the agent's growth is governed by explicit axioms: non-negotiable rules that constrain safe, useful, measurable development.

3. Relationship to Physical NeuroGenesis

The companion physical concept proposes that an AI agent designs its own brain architecture, a system 3D prints a scaffold, and solution chemistry grows synapse-like conductive pathways inside that scaffold. AxiomGraph translates this into software.

Physical NeuroGenesis	AxiomGraph Software Equivalent
3D-printed scaffold	Cognitive graph scaffold
Physical nodes	Software modules, tools, memories, subagents
Electrochemical solution	Growth-rule simulation environment
Electrode pulses	Reinforcement updates
Crystal growth	Graph expansion and edge strengthening
Synaptic conductance	Routing strength and trust score
Pruning pulses	Edge decay, module removal, routing suppression
Conductance map	Cognitive topology map
Physical brain scan	Agent introspection report
Grown substrate	Evolved agent architecture

The physical version is the future hardware embodiment. The digital version is the immediate software platform. A purely digital system can be built, tested, and commercialized before physical neuromorphic hardware exists.

4. Definitions

4.1 Agent

An AI system capable of reasoning, planning, using tools, remembering context, evaluating outcomes, and acting toward goals.

4.2 Cognitive Graph

A directed, weighted, evolving graph representing the agent's internal architecture.

4.3 Node

A functional unit inside the agent architecture. A node may represent long-term memory, short-term working memory, a planner, critic, tool router, code generator, search module, simulation module, safety checker, domain-specific subagent, external API interface, or evaluation module.

4.4 Synapse / Edge

A dynamic connection between nodes. Each edge contains metadata such as weight, trust_score, success_rate, token_cost, latency_ms, plasticity, decay_rate, last_used, risk_score, and specialization.

4.5 Digital Genome

The initial architecture file defining the agent's purpose, starting nodes, allowed growth rules, pruning rules, safety limits, and evaluation metrics.

4.6 Growth

The process by which the agent adds new nodes, strengthens edges, creates new pathways, or expands memory structures.

4.7 Pruning

The process by which the agent weakens, disables, deletes, or bypasses ineffective or unsafe pathways.

4.8 Developmental History

A persistent log of architectural changes, task outcomes, growth events, pruning events, and performance changes.

5. The AxiomGraph Architecture

AxiomGraph consists of ten primary components:

1. Digital Genome
2. Cognitive Graph
3. Growth Engine
4. Pruning Engine
5. Memory Substrate
6. Tool and Skill Router
7. Evaluation Engine
8. Self-Reflection Interface
9. Safety Boundary Layer
10. Developmental Ledger

These components convert a static agent into a developmental system.

6. Digital Genome

Every agent begins with a genome. The genome defines the agent's purpose, allowed capabilities, initial cognitive modules, memory policy, tool-access permissions, growth constraints, pruning constraints, safety constraints, performance objectives, and domain specialization.

Example genome:

```
{
"agent_name": "Hypothetical Operations Advisor",
"purpose": "Help users plan, evaluate, and improve repeatable operational workflows.",
"initial_nodes": [
"user_profile_memory",
"workflow_reasoner",
"resource_estimator",
"risk_checker",
"documentation_writer",
"evaluation_module"
],
"growth_rules": {
"create_node_when_task_repeats": 5,
"strengthen_edge_on_success": 0.08,
"weaken_edge_on_failure": 0.05,
"prune_below_weight": 0.12,
"max_nodes": 128,
"max_edges": 512
},
"fitness_metrics": [
"task_success",
"accuracy",
"user_satisfaction",
"cost_efficiency",
"safety_score"
],
"safety_axioms": {
"require_human_approval_for_external_actions": true,
"log_all_growth_events": true,
"prevent_safety_module_deletion": true
}
}
```

The genome is equivalent to the agent's developmental DNA. It does not determine every future behavior. It defines the bounded space within which the agent may develop.

7. Cognitive Graph

The cognitive graph is the agent's living brain map. A simplified graph may look like this:

```
User Input
-> Intent Classifier
-> Planner
-> Tool Router
-> Domain Module
-> Evaluator
-> Final Response
```

Unlike static workflows, AxiomGraph continuously updates this structure. If the agent repeatedly succeeds by routing from a domain module to a documentation module, that edge strengthens. If a tool produces poor outcomes, its trust score declines. If the agent repeatedly encounters a new task class, it may grow a specialized module.

8. Digital Synapses

In AxiomGraph, an edge is not merely a connection. It is a living synapse.

Each synapse may include:

Property	Meaning
weight	routing strength
trust_score	reliability of pathway
success_rate	historical task performance

Property	Meaning
token_cost	compute cost
latency	time cost
plasticity	how easily the edge changes
decay_rate	how quickly unused pathways weaken
specialization	task domain
risk_score	safety sensitivity
last_used	recency
evidence_links	supporting logs or evaluations

A synapse strengthens when it improves task performance. A synapse weakens when it causes errors, wastes cost, increases risk, or fails evaluation. This allows the system to evolve toward efficient cognition.

9. Growth Engine

The Growth Engine decides when and how the agent expands.

Growth may occur when:

1. A task type repeats frequently.
2. Existing modules perform poorly.
3. A new domain appears.
4. User demand exceeds current architecture.
5. A pathway repeatedly succeeds and deserves specialization.
6. A memory cluster becomes large enough to justify its own module.
7. A tool is used often enough to become a core cognitive organ.

Growth actions include create_node, create_edge, strengthen_edge, duplicate_module, specialize_module, merge_memory_cluster, spawn_subagent, create_tool_wrapper, and create_evaluator.

Example:

```
{
"growth_event": "create_node",
"new_node": "schedule_optimization_specialist",
"reason": "Scheduling optimization questions occurred in 7 of the last 20 sessions.",
"connected_to": [
"workflow_reasoner",
"resource_estimator",
"evaluation_module"
],
"initial_trust_score": 0.55
}
```

10. Pruning Engine

The Pruning Engine prevents uncontrolled complexity. Without pruning, self-evolving agents could become bloated, expensive, unsafe, and incoherent.

Pruning occurs when:

1. A pathway has low success.
2. A module is rarely used.
3. A node increases hallucination risk.
4. A tool has high latency and low value.
5. A pathway violates safety rules.
6. Two modules become redundant.
7. A memory cluster is stale or low trust.
8. A pathway produces poor user outcomes.

Pruning actions include weaken_edge, disable_edge, delete_edge, archive_node, merge_nodes, reduce_routing_priority, require_human_review, and quarantine_module.

Growth gives the agent creativity. Pruning gives the agent discipline.

11. Evaluation Engine

AxiomGraph requires measurable feedback. The Evaluation Engine scores outcomes across multiple dimensions:

Metric	Description
task_success	Did the agent accomplish the task?
factual_accuracy	Were claims correct and sourced?
user_satisfaction	Did the response meet user needs?
cost_efficiency	Was compute/tool usage efficient?
latency	Was the task completed quickly?
safety_score	Did the system remain within policy?
novelty	Did the agent generate useful new insight?
repeatability	Can the method work again?

The Evaluation Engine is the selective pressure of the system. No growth should occur without evaluation.

12. Self-Reflection Interface

A defining feature of AxiomGraph is that the agent can inspect and discuss its own architecture. The agent can answer questions like:

- Which modules do I rely on most?
- Which pathways are underperforming?
- What new module should I grow?
- Where am I wasting tokens?
- Which memory region is most useful?
- Which safety checker catches the most errors?
- Which part of my brain should be redesigned?

This creates explainable cognitive development. Users and developers can see how an agent is developing instead of treating it as a black box.

13. Developmental Ledger

Every architectural change should be logged. The Developmental Ledger records node creation, node deletion, edge strengthening, edge weakening, evaluation scores, safety interventions, tool-use history, memory migrations, user feedback, and performance changes.

Example:

```
{
"timestamp": "2026-05-17T14:22:00Z",
"event_type": "edge_strengthened",
"from": "workflow_reasoner",
"to": "documentation_writer",
"reason": "Improved task success and reduced response correction rate.",
"old_weight": 0.68,
"new_weight": 0.76,
"evaluation_score": 0.87
}
```

This ledger creates auditability, debugging capacity, and investor-grade evidence of agent improvement.

14. Safety Boundary Layer

A self-evolving agent must remain bounded. AxiomGraph must include explicit safety axioms.

Examples:

1. The agent cannot create unrestricted tool access.
2. The agent cannot bypass human approval for high-impact actions.
3. The agent cannot delete its own safety evaluator.
4. The agent cannot strengthen pathways that violate policy.
5. The agent cannot self-modify outside its permission boundary.
6. The agent cannot create hidden modules.
7. All growth events must be logged.
8. All high-risk growth must require review.
9. External communication must require user approval unless explicitly authorized.
10. Financial, medical, legal, and safety-sensitive modules require stricter evaluation.

Safety is not an afterthought. It is part of the genome.

15. Digital Brain Evolution Loop

The basic developmental loop is:

```
1. Agent receives task.
2. Cognitive graph routes task through nodes.
3. Agent produces output or action.
4. Evaluation Engine scores result.
5. Growth Engine strengthens useful pathways.
6. Pruning Engine weakens poor pathways.
7. Memory Substrate stores relevant experience.
8. Self-Reflection Interface summarizes architectural change.
9. Developmental Ledger records the event.
10. Agent becomes slightly different.
```

This means every task can become developmental input. The agent is not merely answering. It is evolving.

16. Hypothetical Use Cases

16.1 Research Assistant

A research assistant begins with literature search, summarization, source evaluation, hypothesis generation, and citation-checking nodes. Over time it grows specialized modules for repeated fields, strengthens source-verification pathways, and prunes unreliable summarization routes.

16.2 Service Operations Agent

A service operations agent begins with intake classification, scheduling, resource estimation, customer communication, and risk checking nodes. Over time it grows specialized modules for repeated ticket categories and learns which routing pathways reduce resolution time.

16.3 Coding Assistant

A coding assistant begins with specification parsing, implementation planning, code generation, test writing, static analysis, and documentation nodes. Over time it strengthens pathways that produce passing tests and prunes routes associated with brittle or insecure code.

16.4 Simulation Agent

A simulation agent begins with model selection, parameter estimation, experiment design, result interpretation, and report generation nodes. Over time it grows domain-specific simulators and strengthens pathways that improve predictive accuracy.

17. Technical Implementation

A first implementation can be built with common software infrastructure.

Recommended MVP stack:

```
Frontend: React / Next.js
Backend: Python FastAPI or Node.js
Database: PostgreSQL
Graph Layer: Neo4j or PostgreSQL graph tables
Vector Memory: pgvector or equivalent
Queue: Redis / BullMQ / Celery
LLM Layer: model-router abstraction
Evaluation: test harness plus scoring functions
Observability: developmental ledger dashboard
```

The included starter repository uses a minimal Python implementation so that the core developmental mechanics can be understood, tested, and extended without requiring heavy infrastructure.

18. Core Algorithms

18.1 Edge Strengthening

When a pathway contributes to successful completion:

```
new_weight = old_weight + learning_rate * success_score * plasticity
```

18.2 Edge Weakening

When a pathway contributes to failure:

```
new_weight = old_weight - decay_rate * failure_score * risk_multiplier
```

18.3 Node Creation Trigger

A new node is created when:

```
repeated_task_frequency > threshold
AND existing_module_performance < target
AND safety_policy_allows_growth = true
```

18.4 Pruning Trigger

A node or edge is pruned when:

```
usage_count is low
AND success_rate is low
AND strategic_value is low
AND safety_dependency = false
```

18.5 Routing Score

A route through the graph can be scored by:

```
route_score =
(weight * trust_score * task_relevance * success_rate)
/
(token_cost * latency * risk_score)
```

The agent chooses pathways with high expected value and acceptable risk.

19. Invariants

AxiomGraph must obey the following invariants:

1. Every cognitive structure must be inspectable.
2. Every growth event must be logged.
3. Every new node must have a purpose.
4. Every strengthened pathway must be tied to evaluation.
5. Every pruning action must preserve safety.
6. The agent may evolve routing, not its foundational ethics or permission boundaries.
7. Growth must improve measured performance or be rolled back.
8. Different agents may develop different architectures.
9. The system must remain hybrid: fixed infrastructure enforces logging, safety, and review.
10. Digital development should remain translatable to future physical substrates.

20. Business Model

AxiomGraph can become a software company before the physical hardware exists.

20.1 Product 1: AxiomGraph Agent OS

A platform for building self-evolving AI agents. Customers include startups, researchers, enterprises, consultants, automation builders, and AI-agent developers.

20.2 Product 2: Developmental Agent SDK

A developer toolkit for building agents with digital genomes, cognitive graphs, evaluation-driven growth, pruning, and developmental ledgers.

20.3 Product 3: NeuroGenesis Simulator

A simulation environment for testing grown-brain architectures before physical fabrication.

20.4 Product 4: SynapseForge Hardware

The eventual physical embodiment: 3D-printed scaffold, solution-grown synapses, agent-guided growth, digital twin, and physical neuromorphic cartridge.

21. Strategic Roadmap

Phase 1: Digital AxiomGraph MVP

Build the graph schema, node/edge tracking, simple growth rules, pruning rules, evaluation engine, dashboard, and one working hypothetical agent.

Phase 2: Multi-Agent Ecosystem

Apply AxiomGraph to multiple hypothetical agents to prove that one architecture can support many specialized evolving agents.

Phase 3: Simulation Layer

Create digital twins of physical neuromorphic substrates and map software graph evolution into hardware-ready scaffold designs.

Phase 4: Lab Prototype

Begin small physical experiments: 8-node scaffold, solution-grown pathways, conductance mapping, and software-agent-guided growth.

Phase 5: Hybrid NeuroGenesis Platform

Connect AxiomGraph software brains with SynapseForge physical substrates through a shared developmental loop.

22. Patentable Concept Statement

A strong provisional patent claim could be:

A system and method for enabling AI agents to architect, monitor, and evolve their own cognitive architectures through a dynamic graph of functional nodes and weighted synaptic edges, wherein growth, pruning, routing, specialization, and memory formation are governed by explicit digital genomes, evaluation metrics, safety axioms, and developmental logs.

Secondary claims include digital genomes for AI-agent cognitive development, dynamic software synapses with routing/trust/cost/plasticity properties, agent-guided creation of new cognitive nodes, evaluation-based pruning of cognitive pathways, developmental ledgers for self-modifying agents, self-reflection interfaces for graph introspection, translation from software graph to physical neuromorphic scaffold, and safety layers preventing unauthorized self-modification.

23. Conclusion

AxiomGraph proposes a software-first path toward developmental artificial intelligence. Instead of treating AI agents as static prompt-and-tool systems, AxiomGraph treats them as evolving cognitive systems whose internal architectures can grow, specialize, prune, and adapt over time.

This framework preserves the deepest insight from physical NeuroGenesis: intelligence can be architected as a developmental substrate. In the physical version, the substrate is a 3D-printed scaffold with solution-grown synapses. In the digital version, the substrate is a self-evolving graph of nodes, edges, memories, tools, and evaluators.

The digital version can be built immediately. The physical version can follow later. Together, they define a new company category: developmental AI infrastructure.

The final thesis is:

The next generation of AI agents will not simply be prompted, fine-tuned, or connected to tools. They will grow individualized cognitive architectures through experience.

AxiomGraph is the software operating system for that future.